Author

Full name: Arya Prashant Warade

Roll number: 23f2001197

Student email: 23f2001197@ds.study.iitm.ac.in

About me: I am currently pursuing a BSc in Data Science from IIT Madras and also studying Electronics and Telecommunication Engineering at PICT, Pune. I am passionate about data science, IoT, and AI/ML. This project helped me improve my skills in backend programming and full-stack development

Description: The goal of this project is to build a multi-user parking management system for 4-wheeler vehicles. The application allows users to search and book parking lots and enables admins to manage parking data. The project focuses on implementing secure login, efficient database design, and a responsive interface.

AI/LLM Used: 20–30%. ChatGPT was used to debug Flask routing issues, clarify SQLAlchemy usage, and draft initial HTML templates. All code is written and verified by me.

Technologies used

Backend:

- **Flask:** A lightweight web framework used to handle routing, templating, and application structure.
- **Flask-SQLAlchemy:** Used to interact with the database using ORM (Object Relational Mapping), making it easier to manage database queries with Python classes.
- **Flask-Login:** Provides user session management, including login/logout functionality and access control based on user authentication.
- **Flask Blueprint:** A built-in Flask feature used to organize the application into reusable modules (like auth, user, admin, parking).

Front-end:

- **HTML & CSS:** Used to structure and style the web pages.
- **Jinja2:** Templating engine used to generate dynamic HTML pages by injecting data from the backend.
- **Bootstrap 5:** Included via CDN in HTML templates, provides a responsive and mobile-friendly layout with pre-built UI components.

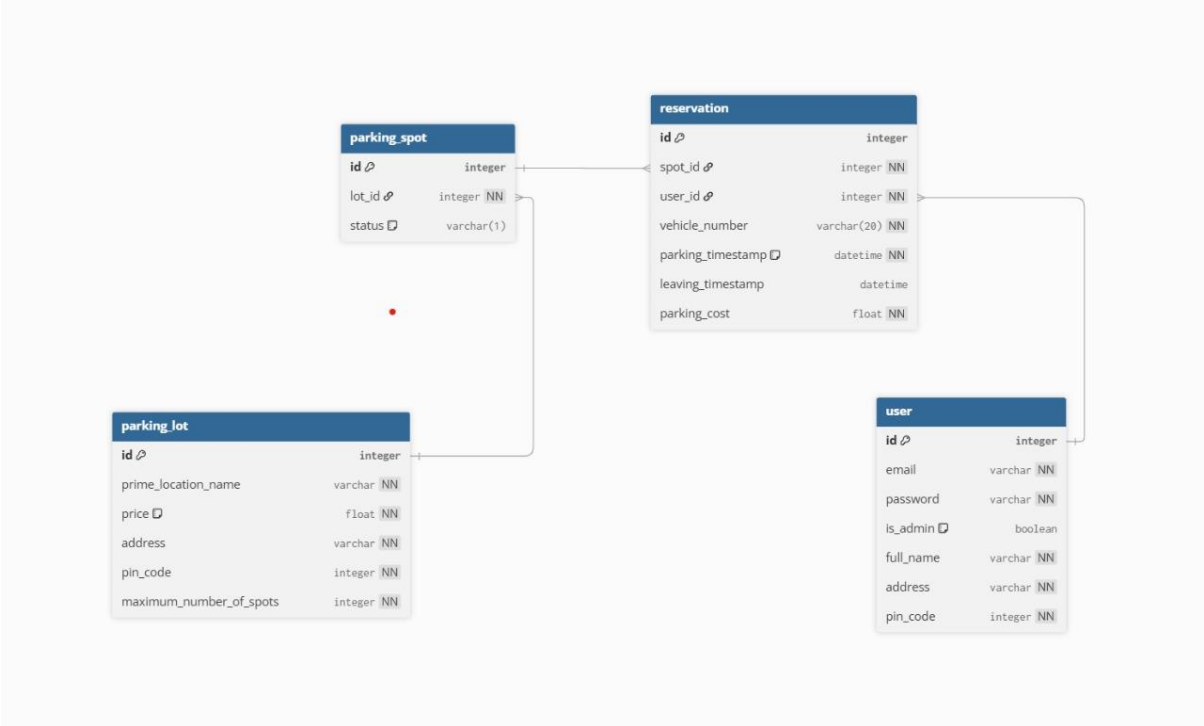
Database:

- **SQLite3:** Integrated with Flask-SQLAlchemy. A lightweight relational database used to store users,parking lots, spots, and reservation records. Suitable for small-to-medium projects and easy to set up.

DB Schema Design

Relationship Overview

- A **User** can have multiple **Reservations** — *One-to-Many*
- A **ParkingLot** contains multiple **ParkingSpots** — *One-to-Many*
- A **ParkingSpot** can be reserved multiple times (over time), each as a **Reservation** — *One-to-Many*



1. **User table:**
    - **Id**: **Primary key** and is **auto-incremented.**
    - **email**: **Unique**, **Not Null** – used for login.
    - **is_admin**: **Boolean,** having a default value of False
    - **password, full_name, address, pin_code**: all **Not Null**..
    - This table is **referenced** by the **Reservation** table via the **user_id** (**FOREIGN KEY)**.
    - An admin user is created during database initialization and bypasses the registration.

2. **ParkingLot table**
    - **Id: PRIMARY KEY** and is AUTO-INCREMENTED.
    - **prime_location_name, price, address, pin_code, maximum_number_of_spots**: all **NOT NULL**
    - Referenced by the ParkingSpot table via **lot_id** (FOREIGN KEY).
    - This table is referenced by the ParkingSpot table via the lot_id FOREIGN KEY.
    - Forms a one-to-many relationship with the ParkingSpot table.

3. **ParkingSpot table**
    - **Id**: PRIMARY KEY and is AUTO-INCREMENTED.
    - **lot_id**: **Foreign Key** referencing ParkingLot(id), **NOT NULL**.

- **status**: **VARCHAR(1)**, default = 'A' (Available); 'O' indicates Occupied.
- status is a VARCHAR(1) indicating spot availability, with default 'A' (Available).
- This table is referenced by the Reservation table via the **spot_id**(FOREIGN KEY).
- Forms a one-to-many relationship with the Reservation table.

4. **Reservation table**
    - **Id**: PRIMARY KEY and is AUTO-INCREMENTED.
    - **spot_id**: FOREIGN KEY referencing ParkingSpot(id) and is NOT NULL.
    - **user_id**: FOREIGN KEY referencing User(id) and is NOT NULL.
    - **vehicle_number**: UNIQUE string field that stores the vehicle registration number.
    - **parking_timestamp**: DATETIME field with a default value of the current time.
    - **leaving_timestamp**: DATETIME field that can be null.
    - **parking_cost**: FLOAT value indicating the total cost.
    - This table connects users to parking spots for each reservation.

Architecture and Features

The system implements both **core and enhanced features** for efficient parking management:

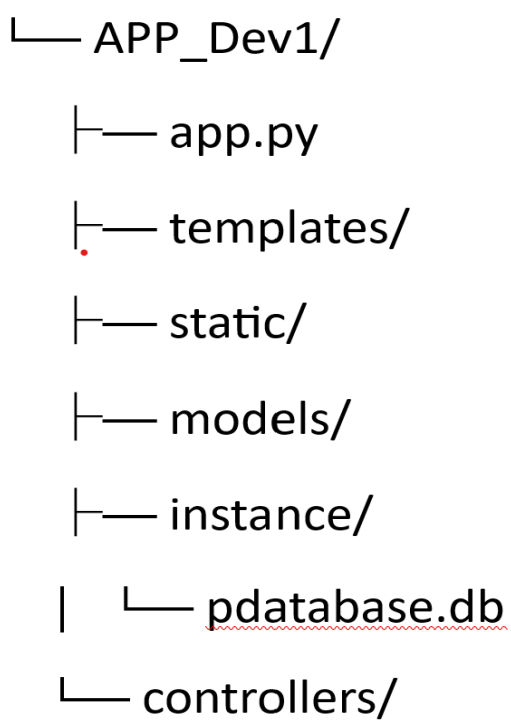
Core Features

- **User Registration & Login**: Managed through Flask-Login, using hashed passwords for security.
- **Admin Dashboard**: Enables admins to create, edit, and delete parking lots; view registered users and manage parking spots.
- **User Dashboard**: Allows users to view their recent parking history and see available lots.
- **Booking System**: Users can book spots; booking updates spot status and stores a reservation record.
- **Session Control**: Access to dashboards and actions is restricted based on user roles using login sessions.

Additional Features

- **Search Functionality**: Users can search parking lots dynamically using keywords such as location names or pin codes. Matching results are displayed in a responsive table.
- **Usage Summary (User)**: A personalized usage summary is shown using charts, indicating how many times each parking lot has been used by the logged-in user.
- **Admin Summary Dashboard**: Admins can view graphical summaries showing revenue per lot, spot availability statistics (occupied/available), and overall lot status (full/empty/partial).
- **Dynamic Availability**: Spot statuses (Occupied/Available) are automatically updated and reflected in the UI to assist users in decision-making.

Folder Structure:



- **root directory (app.py)**: Entry point of the Flask app, registers Blueprints and initializes extensions.
- **controllers/**: Contains route logic split by role — admin, user, auth, and parking management, using Flask Blueprints.
- t**emplates/**: Holds all HTML files, including role-based templates for admin and user interfaces.
- **static/**: Stores CSS files, images, and other static assets used in the front-end.
- **models/**: Defines the database schema using SQLAlchemy, including models like User, ParkingLot, etc.
- **instance/**: Contains the SQLite database file (pdatabase.db) used to store application data.

Video

https://drive.google.com/file/d/1mOCaBgDQdZCl7ACHl55H6qxDss5IbzAw/view?usp=drive_link